

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 256, 256, 64)	640
batch_normalization (BatchNormalization)	(None, 256, 256, 64)	256
max_pooling2d (MaxPooling2D)	(None, 128, 128, 64)	0
conv2d_1 (Conv2D)	(None, 128, 128, 128)	73,856
batch_normalization_1 (BatchNormalization)	(None, 128, 128, 128)	512
max_pooling2d_1 (MaxPooling2D)	(None, 64, 64, 128)	0
conv2d_2 (Conv2D)	(None, 64, 64, 256)	295,168
batch_normalization_2 (BatchNormalization)	(None, 64, 64, 256)	1,024
max_pooling2d_2 (MaxPooling2D)	(None, 32, 32, 256)	0
conv2d_3 (Conv2D)	(None, 32, 32, 512)	1,180,160
batch_normalization_3 (BatchNormalization)	(None, 32, 32, 512)	2,048
conv2d_4 (Conv2D)	(None, 32, 32, 256)	1,179,904
batch_normalization_4 (BatchNormalization)	(None, 32, 32, 256)	1,024
up_sampling2d (UpSampling2D)	(None, 64, 64, 256)	0
conv2d_5 (Conv2D)	(None, 64, 64, 128)	295,040
batch_normalization_5 (BatchNormalization)	(None, 64, 64, 128)	512
up_sampling2d_1 (UpSampling2D)	(None, 128, 128, 128)	0
conv2d_6 (Conv2D)	(None, 128, 128, 64)	73,792
batch_normalization_6 (BatchNormalization)	(None, 128, 128, 64)	256
up_sampling2d_2 (UpSampling2D)	(None, 256, 256, 64)	0
conv2d_7 (Conv2D)	(None, 256, 256, 2)	1,154

Total params: 9,310,408 (35.52 MB)

Trainable params: 3,102,530 (11.84 MB)

Non-trainable params: 2,816 (11.00 KB)

Optimizer params: 6,205,062 (23.67 MB)



Analysis of Output Similarity

```
sample_gray = cv2.cvtColor(load_and_resize(image_paths[0], (256,256)), cv2.COLOR_RGB2GRAY)
L1 = grayscale_to_L_method1(sample_gray)
L2 = grayscale_to_L_method2(sample_gray)
print(f"Max difference between L from method1 and method2: {np.max(np.abs(L1 - L2)):.6f}")
```

```
Max difference between L from method1 and method2: 0.000000
```

Looking at your first image, Method 1 and Method 2 produce almost identical results.

- **Why?** As expected, both methods feed exactly the same information to the model. Both `np.stack((gray,) * 3)` and `cv2.cvtColor(gray, cv2.COLOR_GRAY2RGB)` generate a three-channel image where all channels are equal ($R = G = B$).
- **Colorization Quality:** The model outputs a "plausible" colorization rather than a "perfect" one. It appears to have correctly recognized that the foreground (grass and tree base) should lean toward brown/green. Additionally, it introduced warm reddish-orange tones in the lower left corner, likely an attempt to apply color patterns learned during training.

Note

- The suboptimal colorization in these images illustrates the "ambiguity of color" problem. For any grayscale input, there is no single correct ground-truth color (for example, a mountain might appear at sunrise or at noon). As a result, the autoencoder relies on global statistical priors rather than explicit semantic understanding of objects. When the input lacks distinctive features, the model falls back on "safe" colors such as orange and brown, leading to the observed color bleeding in areas like the barn's base or the mountain slopes.



Analysis of the Rotation Test

The first image reveals how sensitive CNNs are to spatial orientation:

- What we see:** The output from the "Rotation Test" differs noticeably from the "Without Rotation" output.
- Why this happens:** CNNs lack built-in rotation invariance. During training, the model learned to associate colors with specific features (e.g., trees, ground) based on their upright orientation. Rotating the input by 45 degrees presented the network with a "tilted" landscape. Because its convolutional filters are designed to respond to horizontal and vertical edges, they processed the rotated features differently — leading to a distinct color output. After rotating the image back to its original angle, those "incorrectly colored" pixels remained, which explains the visible discrepancy between the two results.



Gaussian Noise Impact on Colorization

- **Low noise ($\sigma = 0.1$):** Model is robust. Output looks slightly grainier, but global color and semantics stay intact.
- **High noise ($\sigma = 3.0$):** Color consistency breaks down. Color bleeding and artifacts appear.

Why?

Low noise gets averaged out by the CNN's receptive fields (implicit denoising). High noise mimics real structures, tricking the decoder into assigning wrong colors.

Affected areas:

- Flat areas (sky/fog) → blotchy artifacts (gradients disrupted).
- Edges/textures → misclassification (e.g., foliage color on bark). The worst semantic errors happen here.

Conclusion:

The model overfit to clean, high-contrast inputs. Training with noisy data would improve robustness.



Brightness Sensitivity (Stability)

- **Observation:** Output depends heavily on input brightness, not just object recognition.
- **Darker input (0.7x):** Colors become more saturated. The model expects certain luminance patterns (e.g., for foliage). Darker inputs trick it into treating low-light textures as brighter regions, causing an over-saturated look.
- **Brighter input (1.3x):** Semantic washout occurs. High brightness clips subtle details the encoder needs. Without clear gradients to guide color mapping, the model defaults to dull, pastel-like colors.
- **Conclusion:** No perceptual constancy here. The model just maps local brightness contrast to color. When brightness falls outside training range, the decoder's non-linearity creates unpredictable artifacts – poor robustness to lighting changes.